SAMPLE REPORT

Database Performance Audit

Example deliverable, based on the public case study github.com/facub/django-perf-demo

Environment

PostgreSQL 16, Django 5.2. 300,000 orders and 900,000 order items. Input received: schema dump, top queries from pg_stat_statements, list of slow pages.

Executive summary

#	Issue	Impact	Effort	Gain
1	Python-side aggregation in report	High	Low	17 s -> 31 ms
2	N+1 queries in orders endpoint	High	Low	535 -> 11 ms
3	Missing partial index (pending)	Medium	Low	9.3 -> 0.04 ms

All three fixes are low effort and return identical results (verified by an automated before/after benchmark). Recommended order: 1, 2, 3.

Finding 2 of 3

N+1 queries in the recent orders endpoint

Symptom

The endpoint takes 535 ms and runs 1,012 queries per request. Each query is fast, so nothing appears in the slow query log; the cost is the number of round trips.

Root cause

```
for order in Order.objects.order_by('-created_at')[:200]:
    order.customer.name          # 1 query per order
    for item in order.items.all():  # 1 query per order
        item.product.name        # 1 query per item
```

Fix (ready to apply)

```
items = OrderItem.objects.select_related('product')
orders = (Order.objects.select_related('customer')
          .prefetch_related(Prefetch('items', queryset=items))
          .order_by('-created_at')[:200])

# migration: index for ORDER BY ... LIMIT
models.Index(fields=['-created_at'], name='order_created_idx')
```

Result

2 queries and 11 ms per request (48x faster), same response body. With the database in another availability zone the gain is larger, because each avoided query also saves network latency.

How to keep it fixed

Add assertNumQueries(2) to the endpoint test so a future change cannot silently reintroduce the N+1.